VBOGS Progress Update

Oakley Thomas

May 2026

Abstract

Variational Bayes Octree Gaussian Splatting (VBOGS) is a mapping pipeline for large outdoor driving scenes. It starts with an Octree-AnyGS scene model, then adds a Variational Bayes Gaussian Splatting (VBGS) uncertainty head that assigns a posterior uncertainty value to each anchor. The current implementation can build an anchor-based Gaussian splatting map from stereo-derived RGB-D observations and render those anchor uncertainties from candidate camera poses. This makes it possible to choose a next-best view (NBV) that is expected to see uncertain scene content. The next stage of the project is to make this uncertainty and NBV loop run in real time on an autonomous vehicle after the initial Octree-AnyGS scene has already been generated.

1 Introduction

Gaussian splatting is a useful representation for autonomous-vehicle mapping because it gives a practical mix of visual quality, rendering speed, and editability. A 3D Gaussian splatting scene is represented as many anisotropic Gaussian primitives that can be rasterized quickly from new viewpoints. For driving data, this is attractive because a reconstructed route can be rendered from perturbed camera poses, inspected in simulation, and edited when rare or safety-critical objects need to be inserted or moved. These properties make Gaussian splatting a strong candidate for building controllable digital twins from logged vehicle data.

For planning, however, a photorealistic reconstruction is not enough. A vehicle also needs to know which parts of the map are poorly constrained and worth observing again. This is the motivation for VBOGS: keep the scalable rendering advantages of Octree-AnyGS, but attach a Bayesian uncertainty estimate to the local regions of the scene that the renderer already uses.

2 Problem Statement and Contributions

VBOGS addresses a gap in standard Gaussian splatting pipelines: they can render a scene well, but they do not directly say how confident the model is about

each visible surface. A candidate view may produce a clean RGB render while still looking at geometry that was seen from only a few angles, reconstructed from noisy stereo, or represented by an ambiguous local fit. For an autonomous vehicle, that distinction matters. A next-best-view planner should prefer poses that reduce map uncertainty, not just poses that show a large amount of already-confident scene content.

The long-term operating mode is an autonomous vehicle that starts with an already generated Octree-AnyGS map and updates uncertainty as new stereo observations arrive. In that setting, the expensive scene reconstruction step can happen offline or during an earlier mapping pass. The online system can then focus on maintaining uncertainty quickly enough to support view selection during vehicle operation.

The main contributions are:

- a per-anchor VBGS uncertainty head for local position-color observations;
- a point bucketing rule that matches the multi-level Octree-AnyGS anchor grid;
- a scalar uncertainty renderer that reuses Octree-AnyGS visibility and level-of-detail selection;
- an alpha-normalized NBV score that favors uncertain visible content instead of simply favoring larger views.

3 System Overview

VBOGS keeps the two upstream systems in distinct roles. Octree-AnyGS is the scene model and renderer. VBGS is the uncertainty model. The connection between them is the anchor grid: stereo points are projected into world coordinates, assigned to Octree-AnyGS anchors, normalized for VBGS fitting, reduced to scalar uncertainty values, and rendered back through Octree-AnyGS for NBV scoring.

The implemented pipeline follows this data flow:

1. prepare KITTI-360 frames into the COLMAP-style layout expected by Octree-AnyGS;
2. train Octree-AnyGS on posed RGB frames to obtain anchors, levels, opacity, geometry, and appearance parameters;
3. estimate stereo depth and export a world-frame point cloud with RGB values;
4. bucket world-frame points into all matching Octree-AnyGS anchor levels;
5. fit one VBGS posterior per sufficiently observed anchor and reduce it to a scalar uncertainty value;

6. render scalar uncertainty from candidate camera poses and select the highest-scoring NBV candidate.

For the planned real-time AV version, this flow splits into two phases. The Octree-AnyGS scene is generated before deployment. During vehicle operation, new stereo frames and poses update the VBGS sufficient statistics, refresh per-anchor uncertainty values, and score planner-provided candidate views without retraining the full Octree-AnyGS scene.

4 Background

4.1 Variational Bayes Gaussian Splatting

Variational Bayes Gaussian Splatting (VBGS) is used here as the uncertainty model. Rather than keeping only point estimates for mixture components, VBGS fits a Bayesian Gaussian mixture model and maintains posterior distributions over the component weights and parameters. In VBOGS, each observation is a six-dimensional vector containing 3D position and RGB color. The useful property is that sparse, noisy, or multi-modal observations naturally lead to broader or more ambiguous posteriors than regions that have been observed consistently.

That posterior information is what makes VBGS useful for next-best-view planning. A normal Gaussian splatting model predicts what a view should look like, but it does not say how much evidence supports the local geometry and color behind that prediction. VBOGS uses VBGS to fill that gap. It does not replace the renderer with VBGS; instead, it fits a VBGS posterior to the normalized samples assigned to each anchor and converts that posterior into one scalar uncertainty value.

4.2 Octree-AnyGS

Octree-AnyGS provides the scene representation. Like standard 3D Gaussian Splatting, it represents a scene with Gaussian primitives that can be rasterized efficiently. Its important addition is an octree-style level-of-detail structure. Scene content is organized into anchors at different spatial scales, so the renderer can use finer support nearby and coarser support farther away. This is a good fit for outdoor driving scenes, where the same map may include close objects, distant buildings, road surfaces, and background structure.

For VBOGS, the anchors are the key interface. Once Octree-AnyGS has been trained from posed RGB imagery, each anchor is treated as the local cell that should receive an uncertainty estimate. Stereo-derived world-space points are bucketed into those cells using the same discretization that Octree-AnyGS used to create them. The corresponding normalized samples are passed to VBGS, producing one posterior and one uncertainty value per anchor. During NBV scoring, the renderer follows the usual Octree-AnyGS geometry and visibility path, but renders uncertainty instead of learned color.

5 Dataset

The current implementation uses the KITTI-360 perspective stereo dataset, with initial experiments built around the synchronized May 28, 2013 drive 0008 sequence. KITTI-360 is a natural starting point because it contains urban driving sequences with calibrated stereo cameras, vehicle poses, and large outdoor structure: roads, buildings, parked vehicles, vegetation, and signs. Those ingredients match the setting VBOGS is meant for, where the map spans a long trajectory and contains scene content at many spatial scales.

Different parts of the dataset are used for different jobs. The posed left-camera RGB frames are converted into the COLMAP-style layout expected by Octree-AnyGS and used to train the main splatting scene. The rectified left and right images are then used to estimate disparity, recover depth, and generate colored 3D points. Camera intrinsics, stereo calibration, and the provided camera-to-world poses place those points in the world frame. Each exported point stores world position, RGB color from the left image, and source frame id in `points_world.npz`.

Stereo is not used as a direct training loss for Octree-AnyGS in the current pipeline. Instead, it supplies the position-color observations used by VBGS after the Octree-AnyGS scene has already been trained. This separation lets the same dataset provide a photometric scene model and an independent observation stream for estimating per-anchor uncertainty.

6 VBOGS Method

The method is built around a simple division of labor. Octree-AnyGS remains responsible for geometry, appearance, visibility, and level-of-detail rendering. VBGS estimates uncertainty for the local anchor regions that Octree-AnyGS already uses. The output is therefore not a separate map, but the original Octree-AnyGS scene with one uncertainty value attached to each anchor.

The pipeline has four main steps. First, Octree-AnyGS is trained in the standard way from posed RGB images, producing anchors, levels, opacities, and appearance parameters. Second, stereo imagery is converted into colored 3D observations. Third, those observations are assigned to Octree-AnyGS anchors and used to fit local VBGS posteriors. Finally, the uncertainty values are rendered from candidate camera poses to select a next-best view.

Stereo data provides the observations for the uncertainty model. For a pixel with disparity $d(u, v)$, focal length f_x, and stereo baseline B, the depth estimate is

$$z(u, v) = \frac{f_x B}{d(u, v)}.$$

The valid depth pixel is unprojected through the camera intrinsics K, transformed into the world frame by the left-camera pose T_{wc}, and paired with its RGB value:

$$p_n^c = z_n K^{-1} \begin{bmatrix} u_n \\ v_n \\ 1 \end{bmatrix}, \qquad p_n^w = T_{wc} \begin{bmatrix} p_n^c \\ 1 \end{bmatrix},$$

$$x_n^w = \begin{bmatrix} p_n^w \\ c_n \end{bmatrix} \in \mathbb{R}^6, \qquad p_n^w \in \mathbb{R}^3, \qquad c_n \in \mathbb{R}^3.$$

The world-frame coordinates are retained for anchor assignment, while the six-dimensional samples are normalized before VBGS fitting:

$$x_n = (x_n^w - \mu_x) \oslash \sigma_x,$$

where μ_x and σ_x are global normalization statistics and $\oslash$ denotes elementwise division. This frame separation is central to the method: Octree-AnyGS anchors live in world space, but the VBGS priors expect normalized data.

6.1 Point-to-Anchor Assignment

This step is the bridge between the two upstream methods. Octree-AnyGS provides a multi-level set of anchors in world space, and VBOGS assigns each stereo point to the same cells that Octree-AnyGS uses for rendering. The grid needs to match exactly; otherwise the uncertainty estimate would be attached to the wrong rendered anchor.

Let a_i denote Octree-AnyGS anchor i, with world position p_i^a and level ℓ_i. The cell size at level ℓ is

$$h_\ell = \frac{\texttt{voxel\_size}}{\texttt{fork}^\ell}.$$

VBOGS uses the same grid discretization as Octree-AnyGS. A point position p is mapped to an integer grid coordinate at level ℓ by

$$g(p, \ell) = \text{round}\left(\frac{p - \texttt{init\_pos}}{h_\ell}\right).$$

Point n is assigned to anchor i when the point and anchor fall in the same grid cell at the anchor's level:

$$n \in \mathcal{I}_i \quad \Longleftrightarrow \quad g(p_n^w, \ell_i) = g(p_i^a, \ell_i).$$

Because Octree-AnyGS renders with level-of-detail selection, this assignment is performed across all anchor levels, not only the finest level. A coarse anchor therefore receives the same observations it may be asked to represent from a distant camera view.

6.2 Per-Anchor VBGS Fitting

After bucketing, each anchor has a local set of normalized position-color samples. VBOGS fits an independent VBGS mixture to each sufficiently observed anchor. The posterior for an anchor then summarizes the evidence available for that part of the scene, keeping uncertainty aligned with the Octree-AnyGS representation.

For each anchor i, VBOGS forms a normalized local dataset

$$X_i = \{x_n : n \in \mathcal{I}_i\}.$$

If $|X_i|$ is below a minimum observation threshold, the anchor is treated as unobserved and assigned a maximum uncertainty value. Otherwise, VBOGS fits a Bayesian mixture with K_i components:

$$\pi_i \sim \mathrm{Dir}(\alpha_i), \qquad z_n \sim \mathrm{Cat}(\pi_i),$$

$$s_n \mid z_n = k \sim \mathcal{N}\left(\mu_{ik}^s, (\Lambda_{ik}^s)^{-1}\right), \qquad c_n \mid z_n = k \sim \mathcal{N}\left(\mu_{ik}^c, (\Lambda_{ik}^c)^{-1}\right),$$

where $s_n = x_n[0:3]$ is the normalized spatial observation and $c_n = x_n[3:6]$ is the normalized color observation. Variational inference approximates the posterior over mixture weights, assignments, and component parameters:

$$q_i(\pi, z, \Theta) = q_i(\pi) \prod_{n \in \mathcal{I}_i} q_i(z_n) \prod_{k=1}^{K_i} q_i(\Theta_{ik}),$$

where Θ_{ik} denotes the spatial and color parameters of component k. The component responsibility for point n is computed from the expected spatial likelihood, expected color likelihood, and expected mixture weight:

$$\gamma_{nk} = q_i(z_n = k) = \frac{\exp(\ell_{nk})}{\sum_{j=1}^{K_i} \exp(\ell_{nj})},$$

$$\ell_{nk} = \mathbb{E}_q[\log p(s_n \mid \Theta_{ik}^s)] + \mathbb{E}_q[\log p(c_n \mid \Theta_{ik}^c)] + \mathbb{E}_q[\log \pi_{ik}].$$

The effective component count and responsibility-weighted spatial mean are

$$N_{ik} = \sum_{n \in \mathcal{I}_i} \gamma_{nk}, \qquad \bar{s}_{ik} = \frac{1}{N_{ik}} \sum_{n \in \mathcal{I}_i} \gamma_{nk} s_n.$$

The number of mixture components starts at K_{init}. Larger mixtures are accepted only when the mean per-point evidence lower bound (ELBO) improves enough:

$$\bar{\mathcal{L}}_i(K) = \frac{1}{|X_i|} \left(\mathbb{E}_q[\log p(X_i, z, \Theta)] - \mathbb{E}_q[\log q(z, \Theta)]\right),$$

$$\bar{\mathcal{L}}_i(K_{\text{next}}) - \bar{\mathcal{L}}_i(K) \geq \tau_{\text{ELBO}}.$$

6.3 Posterior Uncertainty

The fitted posterior contains more information than the planner needs directly: mixture weights, component assignments, spatial parameters, and color parameters. VBOGS compresses this posterior into one scalar per anchor so uncertainty can be rendered through the existing splatting pipeline.

After fitting, VBOGS reduces each anchor posterior to one scalar uncertainty value. The expected mixture weight for component k is

$$\bar{\pi}_{ik} = \mathbb{E}_{q_i}[\pi_{ik}] = \frac{\alpha_{ik}}{\sum_j \alpha_{ij}}.$$

For an observed anchor, the uncertainty is the expected mixture-weighted posterior entropy:

$$U_i = \sum_{k=1}^{K_i} \bar{\pi}_{ik} \left(H_{\mathrm{NW}}(q_{ik}^s) + H_{\Delta}(q_{ik}^c) \right),$$

where H_{NW} is the entropy of the spatial Normal-Wishart posterior and H_{Δ} is the entropy of the color posterior. The complete per-anchor rule is

$$U_i = \begin{cases} U_{\max}, & |X_i| < N_{\min}, \\ \sum_{k=1}^{K_i} \bar{\pi}_{ik} \left(H_{\mathrm{NW}}(q_{ik}^s) + H_{\Delta}(q_{ik}^c) \right), & |X_i| \geq N_{\min}. \end{cases}$$

This produces an uncertainty vector $U \in \mathbb{R}^{N_a}$, where N_a is the number of Octree-AnyGS anchors.

6.4 Uncertainty Rendering and Next-Best View Selection

The final step turns per-anchor uncertainty into a camera-selection objective. Instead of rendering RGB color, VBOGS renders scalar uncertainty through the Octree-AnyGS visibility and rasterization path. Candidate views can then be compared by how much uncertain visible scene content they observe.

To score a candidate camera pose, VBOGS renders uncertainty through the Octree-AnyGS geometry path. Let $R_U(r)$ be the scalar uncertainty image rendered from candidate camera r, and let $R_\alpha(r)$ be the corresponding alpha image. The score is alpha-normalized visible uncertainty:

$$S(r) = \frac{\sum_{u,v} R_U(r)_{uv}}{\sum_{u,v} R_\alpha(r)_{uv} + \epsilon}.$$

The selected next-best view is

$$r^\star = \arg\max_{r \in \mathcal{C}} S(r),$$

where $\mathcal{C}$ is the set of candidate camera poses. This objective favors views that observe uncertain modeled surfaces, rather than views that merely cover many already-confident pixels.

7 Implementation Status

The repository now covers the core VBOGS stages with project-owned entry points. These include KITTI-360 preparation, Octree-AnyGS training, stereo point cloud export, anchor bucketing, per-anchor VBGS fitting, scalar uncertainty computation, uncertainty rendering, NBV scoring, and diagnostic bundling. These pieces live outside the upstream Octree-AnyGS and VBGS submodules, which keeps the borrowed code clean and makes the VBOGS-specific logic easier to maintain.

The system is still mid-implementation. The main interfaces exist, and smoke artifacts have exercised the stages, but the outputs still need scientific validation. Before the results should be treated as experimental evidence, the project needs a full-scene posterior fit, uncertainty reduction, rendered uncertainty visualization, and NBV selection pass on a scene where the expected uncertain regions are already understood.

After that validation pass, the next engineering target is online AV operation. In that version, the Octree-AnyGS scene is loaded as a fixed initial map, while the VBGS uncertainty head updates from newly observed stereo frames. The main work is to make point-to-anchor assignment, posterior or sufficient-statistics updates, scalar uncertainty refresh, and NBV scoring fast enough for planner-provided candidate poses.

8 Evaluation Plan

The first evaluation step is manual posterior inspection. Anchors with dense, consistent observations should have lower uncertainty, while sparse, noisy, or multi-modal anchors should have higher uncertainty. This check is basic, but important: it verifies that the scalar uncertainty still means what the method claims it means.

The next step is to render uncertainty overlays from known and held-out camera views. These overlays should be compared against scene content, source-frame coverage, and visible reconstruction quality. A useful uncertainty map should highlight poorly constrained regions; it should not simply reproduce image brightness, depth, or alpha coverage.

The final offline check is to inspect NBV candidate rankings. The top-scoring poses should point toward uncertain visible content, and the alpha-normalized score should avoid choosing a view only because it contains more rendered pixels. The review artifacts should include diagnostic images, top-K candidate summaries, and notes on any failure modes.

For the real-time target, evaluation also needs runtime measurements. The key numbers are per-frame stereo processing time, point bucketing latency, VBGS update latency, uncertainty rendering time, NBV scoring time, and total delay from a new stereo frame to an updated recommended view. These should be reported separately from reconstruction quality. A method can behave correctly offline and still be unusable on a vehicle if the update loop is too

slow.

9 Limitations

VBOGS currently estimates uncertainty only on existing Octree-AnyGS anchors. If a region of space has no anchor, `render_scalar` cannot render uncertainty there, so the NBV score does not reason about truly unseen empty space. The current method chooses views of uncertain known surfaces, not exploratory views into unmodeled volume.

The uncertainty head is also only as good as the stereo observations it receives. Stereo mismatch, invalid disparity, reflective surfaces, thin structures, and long-range depth errors can all affect the point cloud that is bucketed into anchors. A stronger stereo or depth backend should directly improve the uncertainty estimates.

The component-growth rule is another practical compromise. It uses mean per-point ELBO improvement to decide whether to add mixture components. This is useful during implementation, but it is not a fully calibrated comparison across mixture sizes because the KL term changes with component count. Future versions should test held-out likelihood, BIC, or another validation-based selection rule.

10 Conclusion

VBOGS combines the scalability and level-of-detail rendering of Octree-AnyGS with the local posterior uncertainty provided by VBGS. The result is an Octree-AnyGS scene with one uncertainty value per anchor. Rendering those values from candidate camera poses gives the mapping pipeline a way to reason about where another observation is likely to be useful.

In its intended deployment form, VBOGS operates after an initial Octree-AnyGS scene has already been built. The offline scene supplies geometry and rendering support. The online uncertainty layer helps the vehicle decide where to observe next as it gathers new stereo data.

11 Future Work

The immediate next step is to validate the offline pipeline, then turn the uncertainty and NBV stages into a real-time AV loop. That work includes completing full-scene posterior review, calibrating uncertainty against held-out observations, testing stronger stereo or depth backends, reusing VBGS sufficient statistics for incremental online updates, keeping uncertainty tensors resident on the GPU where possible, integrating planner-produced reachable poses, and measuring end-to-end latency against vehicle sensor and planning rates.

Longer term, the system should be able to update or densify the Octree-AnyGS scene when repeated observations reveal missing structure. It should

also include an empty-space uncertainty term for unexplored volume and fail-safe behavior for cases where stereo depth, pose estimates, or real-time compute budgets are unreliable.